TEMA 1 · MAPA

Hilos: varios camareros trabajando a la vez

Hasta ahora, un solo camarero lo hacía todo, una cosa detrás de otra. Ahora contratamos varios… y aparece un problema nuevo: se pisan.

Proceso e hilo

«La cafetería y sus camareros, que comparten todo.»

pág. 2

Runnable · Thread · start()

«El encargo, el camarero y "¡empieza!".»

pág. 3 · H01

Ciclo de vida

«Contratado → trabajando ↔ esperando → se va a casa.»

pág. 4 · H01

join · sleep · interrupt

«Espero a que acabes · siesta · una nota pidiendo que pares.»

pág. 5 · H02

Condición de carrera + synchronized

«Si dos tocan lo mismo, se pisan. Candado: de uno en uno.»

pág. 6 · H03

Colecciones + comprobar y actuar

«Pregunta y coge en un solo paso: poll().»

pág. 7 · H04 · Cafeteria

> Lo más importante del tema: **el orden de los hilos NO está garantizado**. Lo decide el sistema operativo, y tu programa tiene que funcionar en cualquier orden.

La cafetería y sus camareros



OTRO proceso

su propia memoria

no puede tocar nuestra caja

Todo programa empieza con UN hilo: el main

y desde él puede crear más

Concurrencia = varios camareros avanzan «a la vez».
Compartir memoria es **rápido** (no hay que copiar nada)… pero **peligroso** si dos tocan lo mismo a la vez (pág. 6).

El orden NO está garantizado
Ejecuta `H01` varias veces: «Camarero 1 atiende mesa 1» y «Camarero 2 atiende mesa 1» salen cada vez en un orden distinto.

El encargo, el camarero y «¡empieza!»

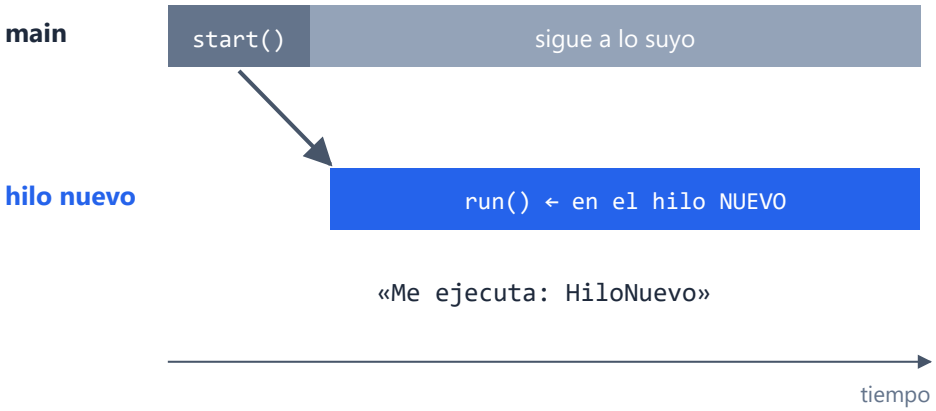
Runnable = el ENCARGO
qué hay que hacer, escrito en `run()`

Thread = el CAMARERO
`new Thread(encargo)`

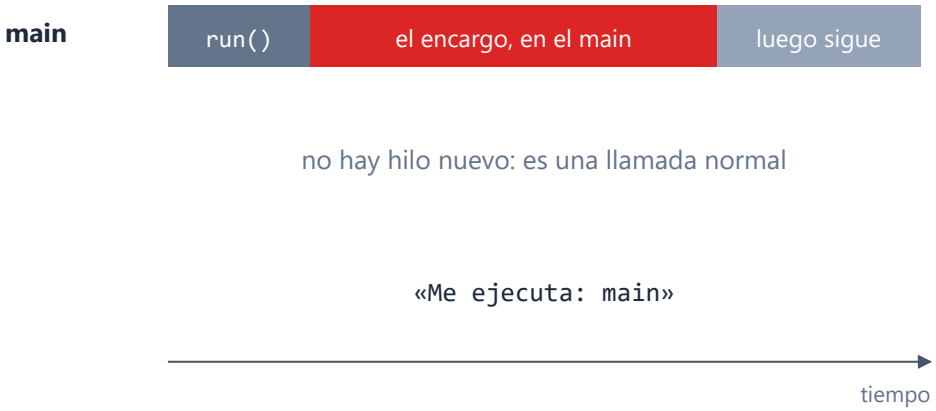
start() = «¡EMPIEZA!»
arranca un hilo nuevo, una sola vez

La trampa: start() frente a run()

✅ t.start(): dos trabajando a la vez



❌ t.run(): lo hace el main solo

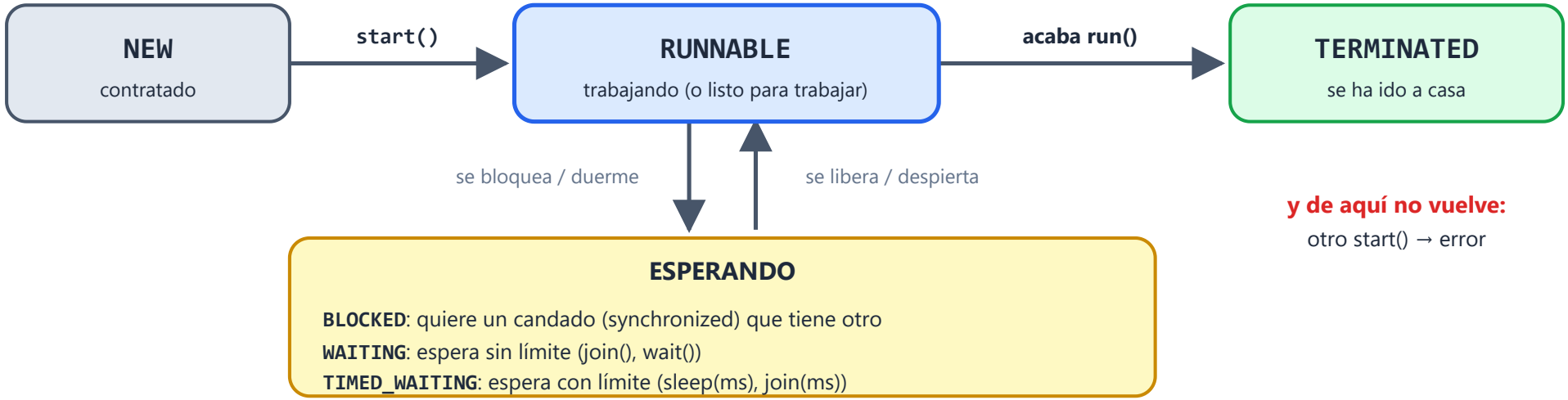



```
class Camarero implements Runnable {   // el encargo
    public void run() { ... }
}
Thread t = new Thread(new Camarero(1));
t.start();                          // ✅
```

start() una sola vez → la 2.ª lanza `IllegalThreadStateException`.

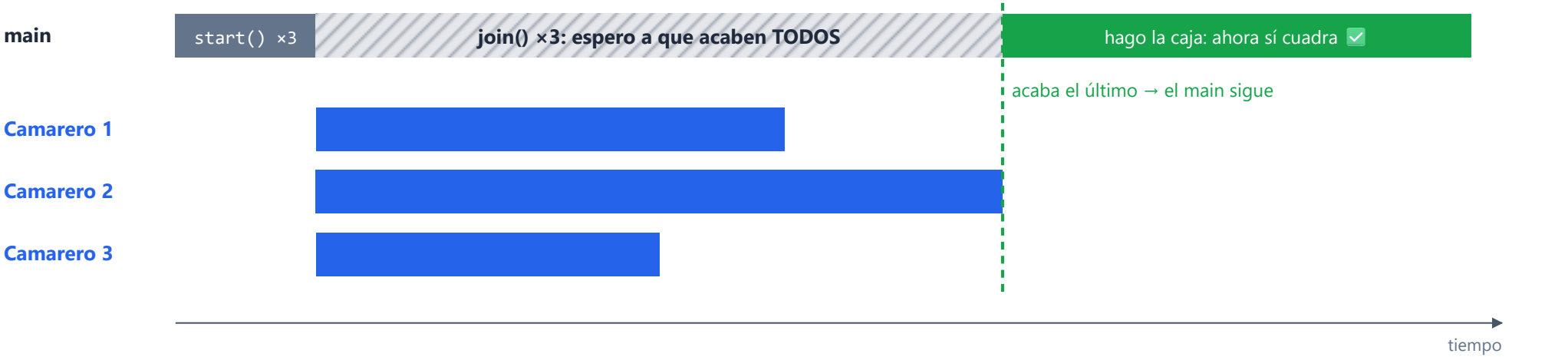
Mejor implements Runnable que extends Thread: solo se puede heredar de una clase.

La vida de un camarero



En H01: «Estado de t1 antes de start(): **NEW**» · «al terminar: **TERMINATED**». Se consulta con `t.getState()` o `t.isAlive()`.

Espero a que acabes · siesta · una nota



join(): espero a que acabes

Sin join, el main hace la caja **antes** de que terminen.

H02: «no he esperado, ¿el hilo sigue vivo? true»

sleep(ms): siesta

Duerme al hilo **que ejecuta esa línea**. Sirve para simular que algo tarda.

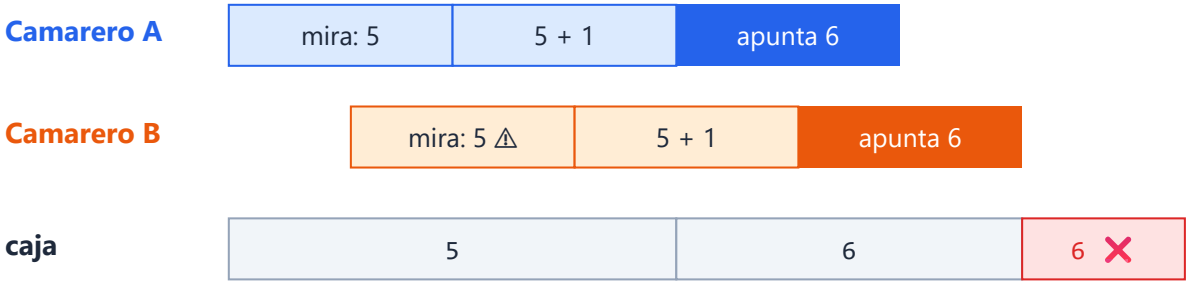
interrupt(): una nota 📝

No lo echa: le pide que pare. Él mira `isInterrupted()` y sale limpio.

H02: «me han interrumpido, salgo limpiamente»

Esquema de siempre: **preparar lo compartido → crear → start() a TODOS → join() a TODOS → resultado**. Nunca start+join en el mismo bucle.

Se pisan… y el candado lo arregla



2 ventas → solo +1: CONDICIÓN DE CARRERA

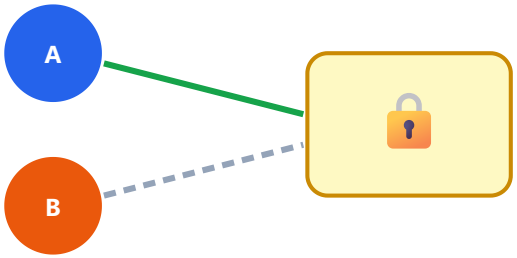
el resultado depende de quién llega antes

H03: esperado 400 000 → sin candado: 125 564

con synchronized: 400 000 ✅

Regla 1 del candado: tiene que ser EL MISMO para todos

✅ un candado compartido (static final)

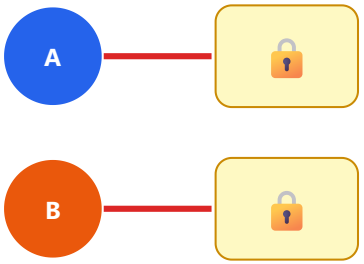


A dentro

B espera

→ de uno en uno

✕ un candado por camarero



cada uno abre el suyo

→ entran los dos a la vez

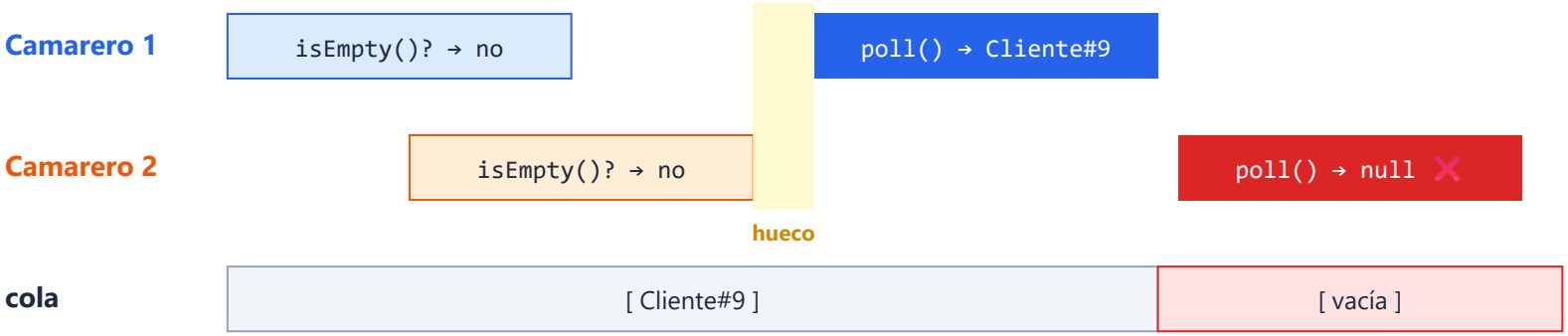
(o: cada camarero con su propio new Caja())

```
private static final Object LOCK = new Object();
synchronized (LOCK) {
    caja++;          // sección crítica: de uno en uno
}                    // se suelta SOLO al salir
```

Regla 2: solo lo imprescindible dentro. Dentro van de uno en uno: si metes todo el `run()`, es como tener un solo camarero.

Pregunta y coge en un solo paso

Cola segura (ConcurrentLinkedQueue) con 1 cliente dentro y 2 camareros. Cada operación es segura… pero dos seguidas no.



Entre COMPROBAR y ACTUAR hay un hueco
y otro se cuela

```
// ❌ dos operaciones → hueco
while (!cola.isEmpty()) {
    String c = cola.poll();      // puede ser null
}

// ✅ una sola operación
String c;
while ((c = cola.poll()) != null) {
    atender(c);
}
```

Normal	Segura para hilos
`LinkedList (Queue)`	`ConcurrentLinkedQueue`
`HashMap`	`ConcurrentHashMap`
`ArrayList`	`CopyOnWriteArrayList`

H04: `ArrayList` con varios hilos: esperado 2000 → **1984**.
`CopyOnWriteArrayList`: **2000** ✅

Esta trampa vuelve en el **tema 3**: entre soltar el readLock y coger el writeLock hay un hueco → hay que volver a comprobar.

El tema 1 en una página

Concepto	Frase clave	Error típico
Proceso / hilo	La cafetería y sus camareros, que comparten todo	Creer que cada hilo tiene sus propios datos
Orden de ejecución	Lo decide el sistema operativo	Suponer que un hilo va antes que otro
`Runnable` / `Thread`	El encargo y el camarero	—
`start()`	«¡Empieza!»: hilo nuevo	Llamar a `run()` · `start()` dos veces
`join()`	Espero a que acabes	`start` y `join` en el mismo bucle · no esperar
`interrupt()`	Una nota pidiendo que pare	Creer que lo mata
Condición de carrera	Dos tocan lo mismo y se pisan	`contador++` sin proteger
`synchronized`	Candado: de uno en uno	Un candado por hilo · meter todo dentro
Colección concurrente	Segura en cada operación, no en dos seguidas	`isEmpty()` + `poll()`

> Del tema 1 al 2: aquí los camareros solo tienen que **no pisarse**. En el tema 2 además tendrán que **coordinarse** (esperarse, turnarse en un pool y devolverse resultados).